Computer Vision Project

End-to-End Deep Learning Pipeline for Visual Recognition

Animal Image Classification Using Transfer Learning

Team Members:

1. Bassant Hossam Salah Eldeen
2. Shahda Mohamed Abd Alkareem
3. Steven Ragy
4. Ahmed Ehab Ahmed Elattar
5. Eyad Hesham Ibrahim
6. Zeyad Wael Helmy
7. Basmalla Helal
8. Basama Ahmed

Under Supervision of:

eng.Habiba Atef

Dr. Shimaa Elgazar

May 7, 2026

1 Introduction

Computer vision systems depend not only on model architecture but also on disciplined data preparation, validation control, and objective testing. This project implements a complete visual recognition workflow for classifying five animal categories: cat, cow, deer, dog, and lion. The main goal is to build a reproducible deep learning pipeline that satisfies academic requirements for training, validation, and final benchmarking while also producing a working application for real-time prediction.

The implementation is based on a local codebase containing three notebooks and one deployment script:

- `1_data_exploration.ipynb`: verifies the dataset structure and visualizes the data.
- `2_model_training.ipynb`: trains a transfer-learning model with validation monitoring.

- `3_evaluation.ipynb`: evaluates the final model on unseen test data.
- `app.py`: provides a Gradio interface for image upload and prediction.

This makes the project not only a modeling exercise, but a full machine learning pipeline from raw data organization to interactive deployment.

2 Dataset

The dataset source is the Kaggle dataset *Animal Image Classification – 5 Species* by `miadul`.[1] It contains five target classes:

- Cat
- Cow
- Deer
- Dog
- Lion

In the local project, the archive was extracted into `data/animals_dataset/` and normalized to match the final code convention:

- split folders: `train`, `val`, `test`
- class folders: `cat`, `cow`, `deer`, `dog`, `lion`

During preprocessing, two naming inconsistencies in the archive were corrected:

- `validation` was renamed to `val`
- `deep` was renamed to `deer`

The actual local extracted archive used in the experiments produced the following split counts:

Split	Cat	Cow	Deer	Dog	Lion	Total
Train	91	86	87	111	89	464
Validation	17	15	17	17	17	83
Test	17	16	16	16	17	82

Table 1: Observed image counts in the normalized local dataset.

Although the Kaggle page describes a simple five-species image classification task and notes a standardized image setup, the local archive inspection showed that the images

[1] https://www.kaggle.com/datasets/miadul/animal-image-classification-5-species

were not perfectly uniform in original dimensions. Therefore, the codebase resizes all inputs to 224 × 224 during training, validation, testing, and deployment for consistency.

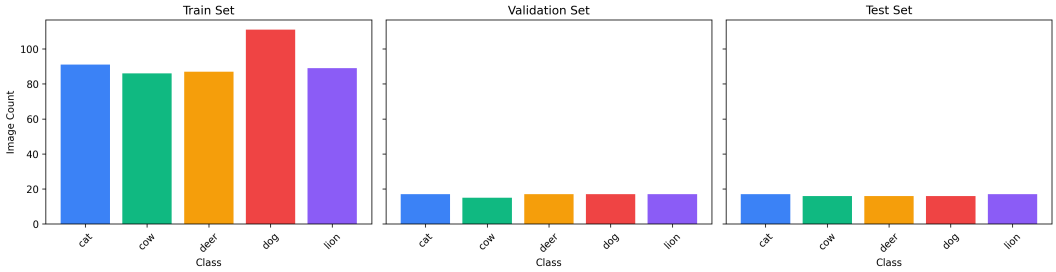

Figure 1: Class distribution across the train, validation, and test splits.

3 Methodology

The project follows the required three-tier methodology: training, validation, and testing.

3.1 Training Phase

The training phase uses a convolutional neural network through transfer learning. Specifically, a pretrained ResNet18 model from `torchvision` was selected because it is lightweight, reliable, and appropriate for a small image dataset. Instead of training from scratch, the pretrained feature extractor was reused and only the final fully connected layer was replaced to match the five project classes.

The training notebook applies the following preprocessing and augmentation steps:

- Resize all images to 224 × 224
- Random horizontal flipping
- Random rotation up to 15 degrees
- Color jitter for brightness, contrast, and saturation
- Tensor conversion
- Normalization using ImageNet mean and standard deviation

The model was trained using:

- Batch size = 16
- Learning rate = 0.001
- Optimizer = Adam
- Loss function = CrossEntropyLoss
- Maximum epochs = 20

These settings were chosen to balance training stability, reasonable speed, and reduced overfitting on a relatively small dataset.

3.2 Validation Phase

The validation split was used at the end of every epoch to monitor generalization. This stage is essential because a model can appear to perform well on training data while failing on unseen data. To address that risk, the training notebook includes two control mechanisms:

- **Learning-rate scheduling**: `ReduceLROnPlateau` lowers the learning rate when validation loss stops improving.
- **Early stopping**: training stops after several non-improving validation epochs, preventing unnecessary overfitting.

The best checkpoint is saved to `models/best_model.pth` whenever the validation loss improves. The checkpoint stores:

- model weights
- class names
- best validation loss
- training history

3.3 Testing Phase

After training was complete, the best saved checkpoint was evaluated on the unseen test split. No random augmentation was used in this phase. Test images were only resized, converted to tensors, and normalized. This keeps final benchmarking unbiased and aligned with standard evaluation practice.

The evaluation notebook computes:

- confusion matrix
- overall accuracy
- precision per class
- recall per class
- F1-score per class
- macro and weighted averages

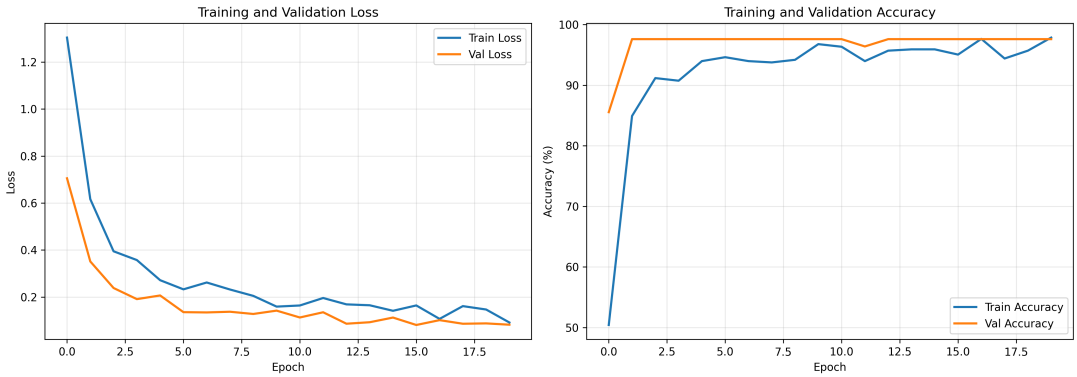

Figure 2: Training and validation loss/accuracy curves produced by the training notebook.

4 Results

The codebase produced all expected result artifacts:

- `class_distribution.png`
- `sample_images.png`
- `training_curves.png`
- `classification_report.txt`
- `confusion_matrix.png`
- `sample_predictions.png`
- `best_model.pth`

The final evaluation report from the local run gave:

- Test accuracy = 97.56%
- Macro precision = 0.9778
- Macro recall = 0.9765
- Macro F1-score = 0.9757

Per-class results are shown in Table 2.

Class	Precision	Recall	F1-score	Support
Cat	1.0000	0.8824	0.9375	17
Cow	1.0000	1.0000	1.0000	16
Deer	0.8889	1.0000	0.9412	16
Dog	1.0000	1.0000	1.0000	16
Lion	1.0000	1.0000	1.0000	17

Table 2: Per-class performance on the unseen test split.

These metrics show excellent overall performance. Three classes achieved perfect precision and recall, while the remaining small errors appeared mainly in cat recall and deer precision. Given the relatively small test split, even one or two mistakes can noticeably affect the class-specific metrics, so these results should be interpreted as strong but still dependent on limited test data size.

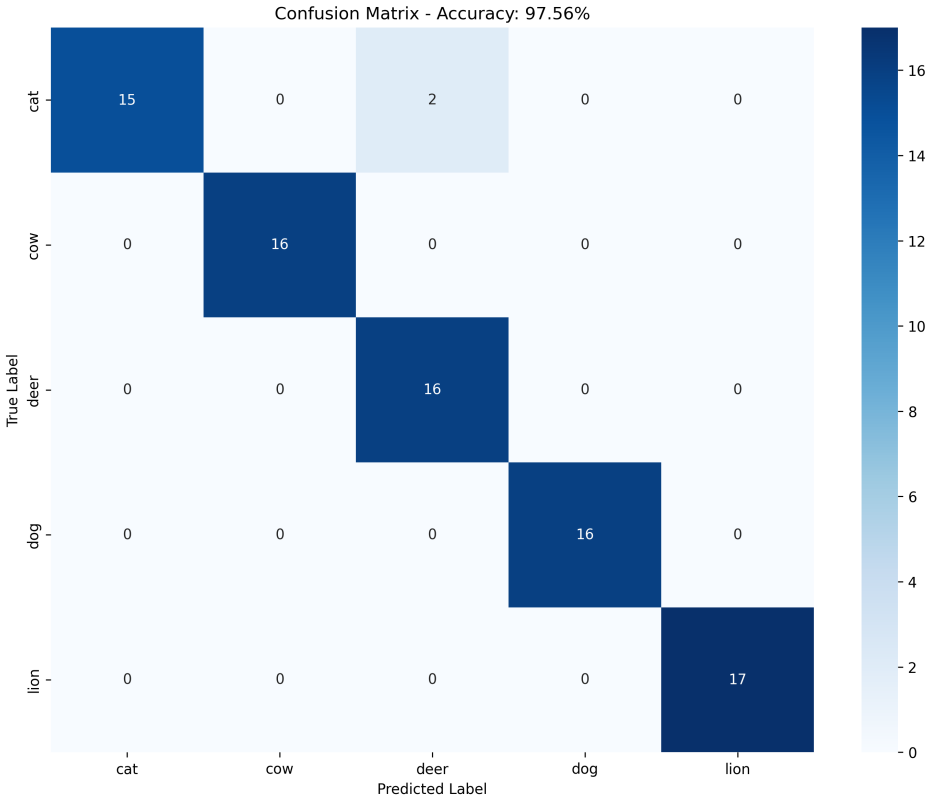

Figure 3: Confusion matrix of the final model on the unseen test split.

5 Review

From a technical perspective, the project satisfies the required pipeline goals and demonstrates good software organization.

Strengths

- The codebase is separated into clear stages: exploration, training, evaluation, and deployment.
- The methodology explicitly uses training, validation, and testing as required.
- Transfer learning made it possible to achieve strong performance on a modest dataset.
- Early stopping and validation-based monitoring reduced overfitting risk.

- The final model is deployable through a working Gradio application.
- The project generates both quantitative and qualitative outputs for interpretation.

Limitations

- The local extracted dataset is not perfectly balanced.
- The test split contains only 82 images, so the final score can shift noticeably with a small number of errors.
- Original image sizes vary, which required resizing before modeling.
- Only one architecture was trained in the final pipeline, so no model comparison study was included.

Deployment Note

The project also includes a Gradio application in `app/app.py`. The app reconstructs the trained ResNet18 classifier, loads the saved checkpoint, preprocesses uploaded images exactly as in evaluation, and returns the predicted class with probability scores. This demonstrates that the project is not only analytically complete but also practically usable.

6 Conclusion

This project successfully implemented an end-to-end deep learning pipeline for visual recognition using an animal image classification dataset. The workflow covered data verification, supervised training, validation-based optimization, final testing, and real-time deployment. The training phase learned class-discriminative visual features through a transfer-learning approach based on ResNet18. The validation phase controlled overfitting through learning-rate scheduling and early stopping. The testing phase provided an unbiased estimate of performance on unseen data using confusion matrix, accuracy, precision, recall, and F1-score.

The final model achieved 97.56% test accuracy with consistently high class-wise performance, showing that the pipeline generalizes well to the available unseen test data. In addition to meeting the academic requirements, the project produced a clean codebase, interpretable results, and a working application suitable for demonstration.